

Complete OOPs (Object-Oriented Programming) Notes — Beginner to Interview-Ready

Har concept ko simple language + real-life example + code example + interview questions ke saath samjhaya gaya hai. Examples mostly Java mein hain (kyunki interviews mein zyada common hai), but concepts har language (Python, C++, C#, JS) pe apply hote hain.

1. What is OOP? (The Big Picture)

Object-Oriented Programming (OOP) ek programming style/paradigm hai jisme hum code ko "**objects**" ke around organize karte hain, instead of just functions and logic.

Real-Life Analogy

Socho tum ek "Car" describe kar rahe ho:

- Car ke **properties (data)** hain: color, brand, speed, fuel level
- Car ke **behaviors (actions)** hain: start(), stop(), accelerate(), brake()

OOP mein, hum is "Car" ko ek **blueprint (Class)** bana dete hain, aur har actual car (Honda City, Swift, etc.) us blueprint se bana ek **Object** hota hai.

```
Class Car (Blueprint)
  - Properties: color, brand, speed
  - Behaviors: start(), stop()

Object 1: myCar = new Car("Red", "Honda")
Object 2: yourCar = new Car("Blue", "Suzuki")
```

Both `myCar` and `yourCar` are objects of class `Car`, but each has its OWN data (different color, brand) — same blueprint, different "instances."

The 4 Pillars of OOP (MOST IMPORTANT — interviewers ALWAYS ask this)

1. **Encapsulation** — Bundling data + methods together, hiding internal details
2. **Inheritance** — One class reusing/extending another class's features
3. **Polymorphism** — Same method behaving differently based on context
4. **Abstraction** — Hiding complex implementation, showing only essential features

We'll cover each in deep detail with examples below — but first, let's understand **Classes and Objects** since everything builds on top of that.

2. Class and Object

Class

A **Class** is a **blueprint/template** — it defines what properties (data/variables) and behaviors (methods/functions) an object will have. **It doesn't occupy memory by itself** — it's just a "design."

Object

An **Object** is an **actual instance** created from a class — it occupies memory and has REAL values for its properties.

Real-Life Analogy

- **Class = Architectural blueprint of a house** (defines: number of rooms, doors, design — but it's just paper, you can't live in a blueprint!)
- **Object = Actual house built from that blueprint** (you can live here, paint it, it has a real address)

You can build MANY houses (objects) from ONE blueprint (class) — each house can have different paint colors, furniture etc. but follows the same basic design.

Code Example (Java)

```
java
```

```
// CLASS = Blueprint
class Car {
    // Properties (also called fields/attributes/instance variables)
    String color;
    String brand;
    int speed;

    // Behavior (method)
    void start() {
        System.out.println(brand + " car is starting...");
    }
}

public class Main {
    public static void main(String[] args) {
        // OBJECTS = actual instances created using "new" keyword
        Car myCar = new Car();
        myCar.color = "Red";
        myCar.brand = "Honda";
        myCar.start();    // Output: Honda car is starting...

        Car yourCar = new Car();
        yourCar.color = "Blue";
        yourCar.brand = "Suzuki";
        yourCar.start();  // Output: Suzuki car is starting...
    }
}
```

Notice: `myCar` and `yourCar` are TWO different objects from the SAME class `Car`, each with its own `color` and `brand`.

Interview Questions — Class & Object

1. What is the difference between a class and an object?

Answer: A class is a blueprint/template that defines properties and behaviors, while an object is a real instance of that class with actual values, occupying memory.

2. Can a class exist without any object?

Answer: Yes — a class is just a definition; it doesn't need an object to exist in code (though it has no real-world effect until objects are created, except for static members).

3. Can multiple objects be created from one class? Do they share data?

Answer: Yes, multiple objects can be created. Each object has its OWN copy of instance variables (unless declared `static`, which is shared across all objects).

3. Encapsulation (Pillar 1)

Encapsulation = Bundling data (variables) and methods (functions) together into a single unit (class), and restricting direct access to some of an object's components (data hiding).

Real-Life Analogy

Think of a **Capsule (medicine)** — the medicine (data) is wrapped inside a protective cover. You can't directly touch the medicine powder — you take the WHOLE capsule.

Or think of an **ATM Machine** — you can't directly access the bank's database/cash; you can only interact through specific buttons/operations (withdraw, check balance) — the internal logic is **hidden** from you.

How is it implemented in code?

1. Make instance variables `private` (so they can't be accessed directly from outside the class).
2. Provide **public** `getter` and `setter` **methods** to read/modify these variables in a controlled way.

Code Example (Java)

```
java
```

```

class BankAccount {
    private double balance; // private = hidden from outside

    // Getter - to READ the value
    public double getBalance() {
        return balance;
    }

    // Setter - to MODIFY the value (with validation/control!)
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Invalid amount!");
        }
    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.deposit(1000);
        // acc.balance = -5000; // ✗ NOT ALLOWED - balance is private!
        acc.withdraw(200);
        System.out.println(acc.getBalance()); // 800.0
    }
}

```

Why is this useful?

Without encapsulation, anyone could write `acc.balance = -999999;` directly and corrupt the data. With encapsulation, ALL changes go through `deposit()`/`withdraw()`, which have **validation logic** — protecting the data's integrity.

Key Terms

- **Data Hiding:** Keeping fields (`private`) so they can't be accessed directly from outside.
- **Getter/Setter:** Public methods to read/write private fields in a controlled way.

🔗 Interview Questions — Encapsulation

1. What is Encapsulation? Give a real-life example.

Answer: Encapsulation is wrapping data and methods into a single unit (class) and restricting direct access to the data using access modifiers (like `private`), exposing controlled access via getters/setters. Example: ATM machine — you can't directly touch the bank's database, only interact via defined operations.

2. How do you achieve Encapsulation in Java?

Answer: By declaring instance variables as `private` and providing `public` getter and setter methods to access/modify them.

3. What is the benefit of using getters/setters instead of public variables directly?

Answer: Getters/setters allow adding validation logic, control read/write access (e.g., read-only fields by providing only a getter), and make future changes to internal implementation easier without breaking external code.

4. Is Encapsulation the same as Data Hiding?

Answer: They're related but not exactly the same — Data Hiding is about restricting access (using `private`), while Encapsulation is the broader concept of bundling data + behavior together AND controlling access. Data hiding is ONE way encapsulation is achieved.

4. Inheritance (Pillar 2)

Inheritance = One class (**child/subclass/derived class**) can **acquire properties and behaviors (fields and methods)** of another class (**parent/superclass/base class**) — promoting **code reusability**.

Real-Life Analogy

Think of **genetic inheritance**: A child inherits features from their parents (eye color, height tendencies) but can also have their OWN unique traits.

Or: "**Vehicle**" is a general concept. "**Car**", "**Bike**", and "**Truck**" are all types of vehicles — they all share common features (have wheels, can move, have an engine) but each also has unique features (Car has 4 doors, Bike has 2 wheels).

Code Example (Java)

```
java

// Parent class / Superclass
class Vehicle {
    String brand;

    void start() {
        System.out.println("Vehicle is starting...");
    }

    void stop() {
        System.out.println("Vehicle has stopped.");
    }
}

// Child class / Subclass — "extends" means inheriting from Vehicle
class Car extends Vehicle {
    int numberOfDoors;

    void openTrunk() {
        System.out.println("Trunk opened.");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.brand = "Honda";           // inherited from Vehicle
        myCar.start();                   // inherited method from Vehicle
        myCar.numberOfDoors = 4;         // Car's own property
        myCar.openTrunk();               // Car's own method
    }
}
```

`Car` got `brand`, `start()`, `stop()` **FOR FREE** from `Vehicle` — without rewriting that code!
`Car` just ADDS its own extra stuff (`numberOfDoors`, `openTrunk()`).

Types of Inheritance (Important!)

Type	Description	Example
Single	One child class inherits from	<code>Car extends Vehicle</code>

Type	Description	Example
Inheritance	ONE parent class	
Multilevel Inheritance	A chain: $A \rightarrow B \rightarrow C$ (C inherits from B, which inherits from A)	<code>Vehicle → Car → SportsCar</code>
Hierarchical Inheritance	ONE parent class, MULTIPLE child classes	<code>Vehicle → Car</code> , <code>Vehicle → Bike</code> , <code>Vehicle → Truck</code>
Multiple Inheritance	One class inherits from MULTIPLE parent classes	⚠ NOT supported directly by classes in Java/C# (to avoid ambiguity) — but achievable via Interfaces
Hybrid Inheritance	Combination of two or more types above	Mix of hierarchical + multilevel

⚠ Important: The "Diamond Problem" (Why Java doesn't allow multiple class inheritance)

Imagine Class `C` inherits from BOTH Class `A` and Class `B`, and BOTH `A` and `B` have a method `display()` with DIFFERENT implementations. When you call `c.display()` — which version should run? A's or B's? This ambiguity is called the "**Diamond Problem**."

Java avoids this by NOT allowing a class to extend multiple classes — but allows a class to implement MULTIPLE interfaces (since interfaces, traditionally, don't have implementation — though `default` methods added some nuance here, which is itself a good interview point!).

Key Terms

- `extends` keyword (Java) — used for inheritance between classes.
- **Parent/Super/Base class** — the class being inherited FROM.
- **Child/Sub/Derived class** — the class that INHERITS.
- `super` keyword — used inside a child class to refer to the parent class (call parent's constructor or methods).

Code Example — `super` keyword

```
java
```



```

class Vehicle {
    String brand;
    Vehicle(String brand) {
        this.brand = brand;
    }
    void info() {
        System.out.println("Brand: " + brand);
    }
}

class Car extends Vehicle {
    int doors;
    Car(String brand, int doors) {
        super(brand); // calls Vehicle's constructor
        this.doors = doors;
    }
    void info() {
        super.info(); // calls Vehicle's info() method
        System.out.println("Doors: " + doors);
    }
}

```

🔗 Interview Questions — Inheritance

1. What is Inheritance? Why is it useful?

Answer: Inheritance allows a class to acquire properties and methods of another class, enabling code reusability and establishing an "IS-A" relationship (e.g., Car IS-A Vehicle).

2. Why doesn't Java support multiple inheritance (with classes)?

Answer: To avoid the "Diamond Problem" — ambiguity when two parent classes have methods with the same signature but different implementations, and the compiler can't decide which one the child should inherit.

3. What's the difference between `super` keyword usages?

Answer: `super(args)` calls the parent class's constructor; `super.methodName()` calls the parent class's version of a method (useful when the child has overridden it but you still want to use the parent's logic too).

4. What is the difference between "IS-A" and "HAS-A" relationships?

Answer: "IS-A" represents inheritance (Car IS-A Vehicle — use `extends`). "HAS-A" represents composition/association (Car HAS-A Engine — Car class has an Engine object as a field, but Engine is not a "type of" Car).

5. Can a subclass access private members of its parent class?

Answer: No — `private` members are NOT accessible to subclasses directly. Subclasses can access `public` and `protected` members of the parent (and `default`/package-private if in the same package).

5. Polymorphism (Pillar 3)

Polymorphism = "Poly" (many) + "morph" (forms) → **One thing existing in MANY forms.** In OOP, it means the **SAME method/function can behave DIFFERENTLY** depending on the object calling it, or the inputs it receives.

Real-Life Analogy

Think of a person who is a **"Teacher"**:

- In a **classroom**, they "teach" students.
- At **home**, they "teach" their kids.
- In a **gym**, as a fitness trainer, they "teach" exercises.

The ACTION is "teach" — but it behaves DIFFERENTLY based on the CONTEXT. Same name, different behavior — that's polymorphism!

Another example: Pressing the **"Power" button** — on a TV it turns the TV on/off, on a phone it locks/unlocks the screen, on a car it starts/stops the engine. Same "action concept," different actual behavior depending on the "object" (device).

Two Types of Polymorphism

A) Compile-Time Polymorphism (Method Overloading)

- Multiple methods with the **SAME NAME** but **DIFFERENT PARAMETERS** (different number/type of arguments) within the SAME class.
- Decided/resolved at **compile time** (the compiler figures out WHICH method to call based on arguments you pass).

Code Example:

```
java
```

```

class Calculator {
    // Method Overloading - same name "add", different parameters
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));           // calls int version → 5
        System.out.println(calc.add(2.5, 3.5));       // calls double version → 6.
        System.out.println(calc.add(1, 2, 3));       // calls 3-param version → 6
    }
}

```

Real-Life Analogy for Overloading: A "print" function on a printer can "print" a document, "print" a photo, or "print" both sides — same word "print," but the printer behaves differently based on WHAT you give it (document vs photo vs double-sided settings). The decision of "which printing process" happens based on what's fed in.

B) Run-Time Polymorphism (Method Overriding)

- A **child class provides its OWN implementation** of a method that is **ALREADY DEFINED** in its **parent class** — same method name, same parameters, but **DIFFERENT** behavior.
- Decided/resolved at **run time** (Java decides WHICH version to run based on the **ACTUAL** object type, not the reference type) — this is called **Dynamic Method Dispatch**.

Code Example:

```
java
```

```

class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof Woof!");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Meow!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog();    // Animal reference, but Dog object
        Animal a2 = new Cat();    // Animal reference, but Cat object

        a1.makeSound(); // Output: Woof Woof! (Dog's version runs)
        a2.makeSound(); // Output: Meow! (Cat's version runs)
    }
}

```

Even though `a1` and `a2` are declared as type `Animal`, Java calls the **ACTUAL object's** (`Dog`/`Cat`) version of `makeSound()` at RUNTIME — this is **Run-time Polymorphism**.

Overloading vs Overriding — Quick Comparison Table (VERY COMMON INTERVIEW QUESTION!)

	Method Overloading	Method Overriding
Where?	Within the SAME class	Between PARENT and CHILD class
Parameters	MUST be different (number/type)	MUST be SAME (same signature)

	Method Overloading	Method Overriding
Resolved at	Compile time (Static Polymorphism)	Run time (Dynamic Polymorphism)
Purpose	Multiple ways to do similar tasks with different inputs	Provide specific implementation in child class
Return type	Can be different	Must be same (or "covariant" — a subtype)
Keyword	None special	<code>@Override</code> annotation (recommended), uses <code>extends</code> /inheritance

Interview Questions — Polymorphism

1. What is Polymorphism? Give a real-life example.

Answer: Polymorphism means "one interface, multiple implementations" — the same method name behaves differently depending on the object or arguments. Example: a "Pay" method could work via UPI, Card, or Cash — same action "Pay," different behavior depending on the payment object used.

2. Difference between Method Overloading and Method Overriding?

Answer: (see table above) Overloading = same class, different parameters, compile-time. Overriding = parent-child classes, same signature, run-time.

3. Can we overload a method by changing only the return type?

Answer: No — if only the return type differs but parameters are the SAME, it's a compile error (ambiguous). Parameters MUST differ for overloading.

4. What is Dynamic Method Dispatch?

Answer: It's the mechanism by which a call to an overridden method is resolved at RUNTIME based on the actual object type (not the reference type) — this is how run-time polymorphism is implemented in Java.

5. Can a static method be overridden?

Answer: No — static methods are bound at compile time (this is called "method hiding" if a subclass defines a static method with the same signature, not true overriding).

6. Abstraction (Pillar 4)

Abstraction = Hiding complex/internal implementation details and showing only the ESSENTIAL features to the user.

Real-Life Analogy

When you drive a car, you use the **steering wheel, accelerator, brake** — you don't need to know HOW the engine combustion works internally, how fuel injection is timed, etc. The car "abstracts away" all this complexity and gives you a SIMPLE interface (steering, pedals) to operate it.

Another example: When you use a **TV remote**, you press "Volume Up" — you don't see/care about the internal circuit that increases the speaker's signal amplitude. The remote gives you a simple ABSTRACTED interface.

How is Abstraction Achieved in Code?

Two ways:

1. **Abstract Classes** (partial abstraction — 0 to 100%)
2. **Interfaces** (full abstraction traditionally — though modern Java allows default methods too)

A) Abstract Class

- A class declared with the `abstract` keyword.
- **CANNOT be instantiated directly** (you can't do `new AbstractClass()`).
- Can have BOTH **abstract methods** (no body — just declaration, MUST be implemented by child classes) AND **regular methods** (with body, can be inherited as-is).

Code Example:

```
java
```

```
abstract class Shape {
    // Abstract method - no implementation, child MUST implement it
    abstract double calculateArea();

    // Regular method - has implementation, shared by all children
    void displayInfo() {
        System.out.println("This is a shape. Area = " + calculateArea());
    }
}

class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }

    @Override
    double calculateArea() {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    double length, width;
    Rectangle(double l, double w) { length = l; width = w; }

    @Override
    double calculateArea() {
        return length * width;
    }
}

public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(4, 6);
        s1.displayInfo(); // This is a shape. Area = 78.53981633974483
        s2.displayInfo(); // This is a shape. Area = 24.0

        // Shape s3 = new Shape(); // ✗ ERROR! Cannot instantiate abstract cl
    }
}
```

`Shape` defines a CONTRACT: "Every shape MUST be able to calculate its area" — but HOW each shape calculates it (circle's formula vs rectangle's formula) is left to the SPECIFIC child class. `Shape` itself doesn't know/care HOW — it just KNOWS that `calculateArea()` will exist.

B) Interface

- A **fully abstract type** (traditionally) — defines WHAT methods a class MUST implement, but NOT how.
- A class uses the `implements` keyword to use an interface.
- A class CAN implement **MULTIPLE interfaces** (this is how Java achieves "multiple inheritance" of behavior, avoiding the Diamond Problem since interfaces traditionally had no implementation).

Code Example:

```
java

interface Drivable {
    void drive();    // no body - just declaration
}

interface Flyable {
    void fly();      // no body
}

// A flying car can BOTH drive AND fly!
class FlyingCar implements Drivable, Flyable {
    @Override
    public void drive() {
        System.out.println("Driving on the road...");
    }
    @Override
    public void fly() {
        System.out.println("Flying in the sky...");
    }
}
```


Abstract Class vs Interface — Quick Comparison Table (CLASSIC INTERVIEW QUESTION!)

	Abstract Class	Interface
Keyword	<code>abstract class</code>	<code>interface</code>
Methods	Can have BOTH abstract AND concrete (implemented) methods	Traditionally only abstract methods (Java 8+ allows <code>default</code> and <code>static</code> methods too)
Variables	Can have instance variables (any access modifier)	Variables are <code>public static final</code> (constants) by default
Inheritance	A class can extend only ONE abstract class	A class can implement MULTIPLE interfaces
Constructor	Can have a constructor	Cannot have a constructor
When to use	When classes share SOME common code/behavior AND have an "IS-A" relationship	When UNRELATED classes need to guarantee certain behavior ("CAN-DO" relationship) — e.g., both <code>Bird</code> and <code>Airplane</code> can <code>fly()</code> , but they're not related by inheritance

Encapsulation vs Abstraction — Another Classic Confusion!

	Encapsulation	Abstraction
Focus	HOW data is protected (implementation level — using private fields + getters/setters)	WHAT functionality is exposed (design level — hiding complexity, showing only essentials)
Real-life example	ATM hides your account data, only lets you access via defined operations (data protection)	ATM shows you simple buttons (Withdraw, Deposit) without showing the internal banking logic (complexity hiding)
Achieved via	Access modifiers (<code>private</code>), <code>getters/setters</code>	Abstract classes, Interfaces

Simple way to remember:

- Abstraction = "WHAT" (hides complexity, focuses on design/interface)

- Encapsulation = "HOW" (hides data, focuses on implementation/security)

🔗 Interview Questions — Abstraction

1. What is Abstraction? How is it different from Encapsulation?

Answer: Abstraction hides implementation complexity and shows only essential features (focus on "what" an object does). Encapsulation hides the internal DATA and protects it via access modifiers (focus on "how" data is protected). (See table above for example.)

2. Difference between Abstract Class and Interface?

Answer: (See table above) — key points: abstract class can have constructors and concrete methods, supports single inheritance; interface traditionally has only abstract methods (plus default/static since Java 8), supports multiple implementation.

3. Can an abstract class have a constructor? If yes, why, if it can't be instantiated?

Answer: Yes, abstract classes CAN have constructors. Even though you can't instantiate the abstract class directly, its constructor is called when a SUBCLASS object is created (via `super()`), to initialize fields defined in the abstract class.

4. When would you use an abstract class vs an interface?

Answer: Use an abstract class when classes share a common base AND some common implementation (IS-A relationship with shared code). Use an interface when you want to define a CAPABILITY/CONTRACT that unrelated classes can implement (CAN-DO relationship), or when you need multiple inheritance of behavior.

5. Can an interface have a method body?

Answer: Since Java 8, interfaces CAN have `default` methods (with a body) and `static` methods — but instance variables in interfaces are still implicitly `public static final` (constants).

7. Constructors

A **Constructor** is a SPECIAL method that is **automatically called when an object is created** — its main job is to **initialize the object's data (fields)**.

Rules of a Constructor

- Has the **SAME NAME** as the class.
- Has **NO return type** (not even `void`).
- Called automatically using the `new` keyword.

Real-Life Analogy

Think of a constructor like the **"setup process" when you unbox a new phone** — it automatically asks you to set language, WiFi, Google account, etc., to get the phone ready for use. You don't manually call this setup — it happens automatically when you "create" (turn on) the phone for the first time.

Types of Constructors

1. Default Constructor (No-Argument Constructor)

If you DON'T write any constructor, Java automatically provides an EMPTY default constructor.

```
java

class Student {
    String name;
    int age;
    // No constructor written -> Java provides: Student() {}
}
```

2. Parameterized Constructor

Takes arguments to initialize fields with SPECIFIC values at the time of object creation.

```
java
```

```

class Student {
    String name;
    int age;

    // Parameterized constructor
    Student(String name, int age) {
        this.name = name; // 'this.name' = the object's field, 'name' = the pa
        this.age = age;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul", 20);
        Student s2 = new Student("Priya", 22);
        System.out.println(s1.name + " is " + s1.age); // Rahul is 20
        System.out.println(s2.name + " is " + s2.age); // Priya is 22
    }
}

```

3. Copy Constructor (concept — not built-in in Java like C++, but commonly implemented manually)

Creates a new object by COPYING values from another object of the same class.

```

java

Student(Student s) {
    this.name = s.name;
    this.age = s.age;
}

```

this Keyword

this refers to the **CURRENT object** (the object on which a method/constructor is being called). Main uses:

1. To differentiate between **instance variables** and **parameters** with the SAME name (as seen above: `this.name = name;`)
2. To call **another constructor** in the SAME class (**constructor chaining**) — `this(args)`
3. To pass the current object as an argument to another method.

Constructor Chaining Example:

java

```
class Student {
    String name;
    int age;
    String course;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
        this.course = "Not Assigned"; // default
    }

    // This constructor calls the above constructor using this(...)
    Student(String name, int age, String course) {
        this(name, age); // calls the 2-arg constructor first
        this.course = course; // then sets course
    }
}
```

Interview Questions — Constructors

1. What is a constructor? How is it different from a method?

Answer: A constructor initializes an object when it's created, has the same name as the class, and has NO return type. A method is a regular function with a return type (which can be `void`) and must be called explicitly.

2. What is a default constructor? Does Java provide one automatically?

Answer: A default constructor is a no-argument constructor. Java automatically provides one ONLY IF you haven't defined ANY constructor in the class. If you define even one parameterized constructor, Java does NOT auto-provide a default one — you'd have to write it yourself if needed.

3. Can a constructor be private? Why would you do that?

Answer: Yes — a private constructor prevents the class from being instantiated from OUTSIDE the class. Commonly used in the **Singleton Design Pattern** (where only ONE instance of a class should ever exist).

4. What is constructor overloading?

Answer: Having multiple constructors in a class with different parameter lists (similar to method overloading) — allows creating objects in different ways.

5. Can constructors be inherited?

Answer: No, constructors are NOT inherited by subclasses (unlike methods). However, a subclass's constructor can call the parent's constructor using `super()`.

8. Access Modifiers

Access Modifiers control the **visibility/accessibility** of classes, methods, and variables — i.e., WHO can access them.

Real-Life Analogy: Levels of Privacy in a House

- **private** = Your personal bedroom — only YOU (the same class) can enter.
- **default** (**package-private**) = Common rooms shared with FAMILY (same package/folder) — family members can enter, outsiders can't.
- **protected** = Like family + close relatives — same package AND any "family members" (subclasses), even if they live elsewhere (different package).
- **public** = Front lawn — ANYONE can access, from anywhere.

Java Access Modifiers Table

Modifier	Same Class	Same Package	Subclass (different package)	Different Package (non-subclass)
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Practical Example

```
java

public class Employee {
    private String ssn;           // only accessible within Employee class
    String department;           // accessible within same package
    protected double salary;     // accessible in same package + subclasses anywhere
    public String name;          // accessible from anywhere
}
```

1. Why do we use `private` for fields and `public` for methods (general best practice)?

Answer: This follows Encapsulation — keep data hidden/protected (`private`), but expose controlled access through `public` getter/setter methods, allowing validation and flexibility to change internal implementation later.

2. What is the difference between `protected` and `default` access?

Answer: `default` (no modifier) allows access only within the SAME PACKAGE. `protected` allows access within the same package PLUS subclasses in DIFFERENT packages too.

3. Can a `private` method be overridden?

Answer: No — `private` methods are not visible to subclasses at all, so they can't be overridden. If a subclass defines a method with the same name, it's treated as a completely new/separate method, not an override.

9. Static Keyword

`static` means a member (variable, method, or block) **belongs to the CLASS itself, NOT to any specific object** — it's SHARED across ALL objects/instances.

Real-Life Analogy

Think of a **School**:

- Each **student** has their OWN name, roll number (instance variables — unique per object).
- But the **school's name** ("DPS Public School") is the SAME for ALL students — it doesn't make sense for EACH student to have their own separate copy of "school name." This is like a `static` variable — ONE COPY, SHARED by everyone.

Code Example — Static Variable

```
java
```

```

class Student {
    String name;
    static String schoolName = "DPS Public School"; // shared by ALL students

    Student(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Rahul");
        Student s2 = new Student("Priya");

        System.out.println(s1.schoolName); // DPS Public School
        System.out.println(s2.schoolName); // DPS Public School

        // Changing it through ONE object affects ALL (since it's shared)
        Student.schoolName = "New Modern School";
        System.out.println(s1.schoolName); // New Modern School
        System.out.println(s2.schoolName); // New Modern School
    }
}

```

Static Methods

- Belong to the class, can be called WITHOUT creating an object:
`ClassName.methodName()`
- CANNOT access non-static (instance) variables/methods directly (because they don't belong to any specific object).
- Common example: `Math.sqrt(25)`, `Math.max(3, 7)` — these are static methods, you don't need to create a `Math` object.

java


```

class MathUtils {
    static int square(int x) {
        return x * x;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(MathUtils.square(5)); // 25 - no object created!
    }
}

```

Static Block

A block of code that runs ONCE, automatically, when the class is FIRST LOADED into memory — often used for one-time initialization of static variables.

```

java

class Config {
    static String apiKey;
    static {
        apiKey = "ABC123"; // runs once when class is loaded
        System.out.println("Static block executed - Config loaded");
    }
}

```

🔗 Interview Questions — Static

- What is the difference between a static and an instance (non-static) variable?**
 Answer: An instance variable has a SEPARATE copy for EACH object; a static variable has ONE SINGLE copy SHARED across ALL objects of the class.
- Why can't a static method access instance variables directly?**
 Answer: Static methods belong to the CLASS and can be called WITHOUT any object existing — but instance variables belong to specific OBJECTS. Since there might be no object (or multiple objects with different values), the static method has no way to know WHICH object's instance variable to use.
- Can we override a static method?**
 Answer: No — static methods are resolved at COMPILE TIME based on the reference type (this is called "method hiding," not overriding, if a subclass declares a static method with the same signature).
- What is a static block used for? When does it run?**

Answer: A static block runs ONCE, automatically, when the class is first loaded into the JVM (before main() or any object creation) — typically used to initialize static variables or perform one-time setup like loading configuration.

10. Association, Aggregation, and Composition (HAS-A Relationships)

Apart from Inheritance (IS-A), classes can be related via **HAS-A** relationships — meaning one class CONTAINS/USES another class as a field. There are 3 levels, based on how STRONG/independent this relationship is.

1. Association

A general **"uses-a"** relationship between two INDEPENDENT classes — both can exist without each other.

Example: A `Teacher` and a `Student` — they are associated (teacher teaches student), but BOTH can exist independently (a teacher exists even without a specific student, and vice versa).

2. Aggregation (Weak "HAS-A")

One class contains a reference to another, but **BOTH can exist independently** — the "contained" object's life is NOT dependent on the "container."

Real-Life Analogy: A `Department` HAS `Professors`. If the `Department` is dissolved/removed, the `Professors` STILL EXIST (they can join another department) — they are NOT destroyed along with the department.

```
java

class Professor {
    String name;
    Professor(String name) { this.name = name; }
}

class Department {
    List<Professor> professors; // Department HAS-A list of Professors
    Department(List<Professor> professors) {
        this.professors = professors;
    }
}
```

3. Composition (Strong "HAS-A")

One class contains another, and the "contained" object's life is **STRICTLY DEPENDENT** on the "container" — if the container is destroyed, the contained object is ALSO destroyed.

Real-Life Analogy: A House HAS Rooms. If the House is demolished, the Rooms CANNOT exist independently — they're destroyed too. Rooms have NO meaning/existence outside of the house they belong to.

```
java
class Room {
    String type;
    Room(String type) { this.type = type; }
}

class House {
    private List<Room> rooms; // Rooms created INSIDE House, tied to its lifecycle
    House() {
        rooms = new ArrayList<>();
        rooms.add(new Room("Bedroom"));
        rooms.add(new Room("Kitchen"));
    }
}
```

Quick Comparison Table

	Association	Aggregation	Composition
Relationship strength	Weakest (general "uses")	Weak "HAS-A"	Strong "HAS-A"
Dependency	Both independent	Both independent (one "has" the other, but neither owns the other's lifecycle)	Contained object's lifecycle DEPENDS on the container
Example	Teacher - Student	Department - Professor	House - Room, Car - Engine
Symbol (UML)	Plain line	Hollow diamond	Filled diamond

Composition vs Inheritance — IMPORTANT Interview Topic!

■ "Favor Composition over Inheritance" is a famous software design principle.

Why? Inheritance creates TIGHT coupling — a child class is heavily dependent on its parent's implementation, and changes in the parent can BREAK the child ("fragile base class problem"). Composition is more FLEXIBLE — you can change behavior by swapping out the "contained" object, without rigid class hierarchies.

Example: Instead of `ElectricCar extends Car extends Vehicle` (deep, rigid hierarchy), you could do:

```
java

class Car {
    private Engine engine; // Composition: Car HAS-A Engine
    Car(Engine engine) {
        this.engine = engine;
    }
}

// Now you can easily create a Car with a PetrolEngine OR ElectricEngine
// by just passing a different Engine object - very flexible!
```

🔗 Interview Questions — Association/Aggregation/Composition

1. What is the difference between Aggregation and Composition?

Answer: In Aggregation, the contained object can exist INDEPENDENTLY of the container (e.g., Professor exists even if Department is removed). In Composition, the contained object's lifecycle is TIED to the container (e.g., Room can't exist without House).

2. What does "favor composition over inheritance" mean, and why?

Answer: It means prefer building classes by COMBINING smaller objects (HAS-A) rather than creating deep inheritance hierarchies (IS-A), because composition is more flexible, avoids tight coupling, and avoids issues like the fragile base class problem.

3. Give an example of IS-A vs HAS-A relationship.

Answer: `Dog extends Animal` is IS-A (Dog IS AN Animal — Inheritance). `Car` having an `Engine` field is HAS-A (Car HAS AN Engine — Composition).

11. `final` Keyword

`final` means "cannot be changed/extended further" — it can be applied to variables, methods, and classes, with different meanings:

Used with	Meaning
<code>final</code> variable	Value CANNOT be changed once assigned (becomes a CONSTANT)
<code>final</code> method	CANNOT be overridden by subclasses
<code>final</code> class	CANNOT be extended/inherited by any other class

Real-Life Analogy

- `final` variable = Your **date of birth** — set once, never changes.
- `final` method = A company's **core security policy procedure** — branches (subclasses) must follow it EXACTLY, can't change/override it.
- `final` class = A **sealed product design** — no one is allowed to create a "modified version"/subclass of it.

Code Example

```
java

final class Constants { // cannot be extended
    static final double PI = 3.14159; // cannot be reassigned
}

class Animal {
    final void breathe() { // cannot be overridden
        System.out.println("Breathing...");
    }
}
```

Famous real example: The `String` class in Java is `final` — you cannot create a subclass of `String`. This ensures `String`'s behavior (especially around immutability and security) can never be altered.

🔗 Interview Questions — final

1. What happens if you try to change a `final` variable's value after initialization?
Answer: Compile-time error — `final` variables can be assigned a value only ONCE (either at declaration or in the constructor for instance finals), and any further reassignment is not allowed.
2. Why is the `String` class `final` in Java?

Answer: To ensure immutability and security — preventing subclasses from overriding String's behavior (e.g., `equals()`, `hashCode()`), which could break things like HashMap keys, security checks, caching (String pool), etc.

3. Can a `final` method be overloaded?

Answer: Yes! `final` prevents OVERRIDING (in subclasses), but overloading (same class, different parameters) is still allowed.

12. The `Object` Class & Important Methods (`toString`, `equals`, `hashCode`)

In Java, EVERY class **implicitly extends** `Object` (even if you don't write `extends Object`) — so every object automatically gets some default methods.

1. `toString()`

- Returns a **String representation** of the object.
- Default version returns something like `ClassName@hashcode` (not very readable) — usually you **override** it to print meaningful info.

java

```
class Student {
    String name;
    int age;
    Student(String name, int age) { this.name = name; this.age = age; }

    @Override
    public String toString() {
        return "Student{name='" + name + "', age=" + age + "}";
    }
}

// System.out.println(new Student("Rahul", 20));
// Output: Student{name='Rahul', age=20} (instead of default Student@1b6d3586)
```

2. `equals()`

- Used to check if TWO OBJECTS are "**equal**" — by default, `equals()` checks if both references point to the SAME object in memory (reference equality, same as `==`).
- Usually **overridden** to compare based on FIELD VALUES (logical/content equality) instead.

Real-Life Analogy: Two different ₹500 notes (different physical objects/serial numbers) — are they "equal"? By DEFAULT (`==`), no (different objects). But if we define "equal" as "same monetary value," then YES — we'd override `equals()` to compare VALUES, not memory addresses.

```
java

class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Point)) return false;
        Point p = (Point) obj;
        return this.x == p.x && this.y == p.y;
    }
}

// Point p1 = new Point(1, 2);
// Point p2 = new Point(1, 2);
// p1 == p2          -> false (different objects in memory)
// p1.equals(p2)     -> true  (same x and y values, after overriding equals)
```

3. `hashCode()`

- Returns an integer "hash" value for an object — used internally by data structures like `HashMap`, `HashSet` to quickly locate/store objects.
- IMPORTANT RULE:** If you override `equals()`, you should ALSO override `hashCode()` — because **two objects that are `.equals()` MUST have the SAME `hashCode()`** (this is required for correct behavior in `HashMaps/HashSets` — otherwise "equal" objects might be stored in different "buckets" and not be found correctly).

Interview Questions — Object Class Methods

1. Difference between `==` and `.equals()`?

Answer: `==` checks REFERENCE equality (do both variables point to the SAME object in memory?). `.equals()` (when overridden) checks LOGICAL/content equality (do both objects have the SAME field values, even if they're different objects in memory?).

2. Why must `hashCode()` be overridden if `equals()` is overridden?

Answer: Because the contract requires that EQUAL objects (per `.equals()`) must produce the SAME hashCode — otherwise, hash-based collections like `HashMap`/`HashSet` may fail to find/store objects correctly (e.g., two "equal" objects might end up in different hash buckets, leading to bugs like duplicate entries or failed lookups).

3. Why do we override `toString()`?

Answer: To provide a meaningful, human-readable string representation of an object (useful for debugging/logging), instead of the default `ClassName@hashcode` format.

13. Quick Summary & Recap

The 4 Pillars — One-Line Recap with Analogy

Pillar	One-Line Meaning	Analogy
Encapsulation	Bundle data + methods, hide internal data	ATM machine — hides bank data, controlled access via buttons
Inheritance	Child class reuses parent class's code	Child inherits traits from parents
Polymorphism	Same method, different behaviors	"Teach" action — different in classroom vs gym
Abstraction	Hide complexity, show only essentials	Car steering wheel — you don't see engine internals

Other Core Concepts Recap

Concept	One-Line Meaning
Class	Blueprint/template
Object	Real instance of a class, occupies memory
Constructor	Special method to initialize objects, same name as class, no return type
<code>this</code>	Refers to current object
<code>super</code>	Refers to parent class (constructor/methods)

Concept	One-Line Meaning
Method Overloading	Same name, different parameters, SAME class (compile-time)
Method Overriding	Same name & parameters, parent vs child class (run-time)
Access Modifiers	<code>private</code> , <code>default</code> , <code>protected</code> , <code>public</code> — control visibility
<code>static</code>	Belongs to class, shared across all objects
<code>final</code>	Cannot be changed (variable) / overridden (method) / extended (class)
Association/Aggregation/Composition	"HAS-A" relationships, varying in strength of dependency
<code>equals()</code> / <code>hashCode()</code> / <code>toString()</code>	Methods from <code>Object</code> class, usually overridden for custom logic

14. Final Master List: 30 Most Common OOP Interview Questions

1. What is OOP? Why use it over procedural programming?
2. What are the 4 pillars of OOP? Explain each with examples.
3. Difference between Class and Object?
4. What is Encapsulation? How is it implemented?
5. What is the benefit of getters/setters over public fields?
6. What is Inheritance? What is the "IS-A" relationship?
7. Why doesn't Java support multiple inheritance via classes?
8. What is `super` keyword used for?
9. Difference between method overloading and overriding?
10. What is run-time vs compile-time polymorphism?
11. Can static methods be overridden? Why/why not?
12. What is Abstraction? How is it different from Encapsulation?
13. Difference between Abstract class and Interface?
14. Can an abstract class have a constructor?
15. What is a constructor? Types of constructors?

16. What is constructor chaining? How is `this()` used?
17. Can constructors be private? Why (Singleton pattern)?
18. Explain all 4 access modifiers with an example.
19. What is the `static` keyword? Give examples of static variable, method, block.
20. What is the difference between instance and static variables?
21. What is the difference between Association, Aggregation, and Composition?
22. Why is "favor composition over inheritance" a good practice?
23. What does the `final` keyword do for variables, methods, and classes?
24. Why is the `String` class `final` in Java?
25. Difference between `==` and `.equals()`?
26. Why should you override `hashCode()` if you override `equals()`?
27. What is the purpose of overriding `toString()`?
28. What is the Diamond Problem, and how is it avoided?
29. What is an "IS-A" vs "HAS-A" relationship? Give examples.
30. Can you give a real-life example combining ALL 4 pillars of OOP in one scenario?

Sample Answer: Think of a "Banking System":

- **Encapsulation:** `Account` class keeps `balance` private, exposes `deposit()`/`withdraw()` methods.
- **Inheritance:** `SavingsAccount` and `CurrentAccount` both `extend` a common `Account` class, reusing common fields/methods.
- **Polymorphism:** Each account type can `calculateInterest()` differently (overriding) — Savings has interest, Current might not.
- **Abstraction:** The `Account` class might be `abstract`, defining WHAT every account must support (`calculateInterest()`), without specifying HOW each type does it.

End of OOP notes! Practice: try writing the `Shape` (Circle/Rectangle), `BankAccount`, and `Animal` (Dog/Cat) examples from scratch yourself without looking — that's the BEST way to make sure you truly understand these concepts before an interview. All the best! 🚀